

Semaine 4 — SQL basique & Mini-projet final

Outils : PostgreSQL · DBeaver · CSV imports

Objectifs de la semaine

Compétence	Ce que tu sauras faire
Charger des données	Importer un CSV dans PostgreSQL via DBeaver
SELECT / WHERE	Interroger et filtrer une table
GROUP BY + agrégats	Reproduire tes groupby() Pandas en SQL
ORDER BY / LIMIT	Trier et isoler les top N
Mini-projet	Analyse complète Coffee Sales en SQL pur

Mise en place — Environnement (Jour 1)

Étape 1 — Vérifier que PostgreSQL tourne

Dans le terminal (PowerShell ou cmd) :

```
psql -U postgres
```

Si erreur « psql not found »

PostgreSQL n'est pas dans le PATH. Ajoute C:\Program Files\PostgreSQL\{version}\bin dans les variables d'environnement.

Étape 2 — Créer une base de données dédiée

Dans psql ou dans DBeaver (onglet SQL Editor) :

```
CREATE DATABASE mois1_data;
```

Dans DBeaver : clic droit sur la connexion → Create → Database → nom : mois1_data.

Étape 3 — Créer les tables et importer les CSV

Règle importante

PostgreSQL ne lit pas les types automatiquement comme Pandas. Tu dois déclarer les colonnes manuellement avant d'importer.

Table coffee_sales

```
CREATE TABLE coffee_sales (  
    date          DATE,  
    datetime      TIMESTAMP,  
    cash_type     VARCHAR(20),  
    card          VARCHAR(50),  
    money         NUMERIC(10, 2),  
    coffee_name   VARCHAR(100)  
);
```

Import via DBeaver : clic droit sur la table → Import Data → choisir le CSV → Séparateur : , —
Encodage : UTF-8 → Mapper les colonnes → Start.

Table transilien

```
CREATE TABLE transilien (  
    date          DATE,  
    annee         INTEGER,  
    gare          VARCHAR(100),  
    type_jour     VARCHAR(5),  
    tranche_horaire VARCHAR(20),  
    voyageurs     INTEGER,  
    mois          INTEGER,  
    jour_semaine  VARCHAR(20)  
);
```

Note : les colonnes mois et jour_semaine ont été extraites en Pandas — inclure si présentes dans ton CSV nettoyé, sinon les calculer en SQL (voir exercice 2).

Table tmdb_movies

```
CREATE TABLE tmdb_movies (  
    id            INTEGER,  
    title         VARCHAR(255),  
    vote_average  NUMERIC(4, 2),  
    vote_count    INTEGER,  
    release_date  DATE,  
    revenue       BIGINT,  
    runtime       INTEGER,  
    popularity    NUMERIC(10, 4),  
    overview      TEXT  
);
```

Étape 4 — Vérifier le chargement

```
SELECT COUNT(*) FROM coffee_sales;    -- attendu : 3636
SELECT COUNT(*) FROM transilien;      -- attendu : 7328
SELECT COUNT(*) FROM tmdb_movies;     -- attendu : 8820

SELECT * FROM coffee_sales LIMIT 5;    -- équivalent df.head()
```

Bloc 1 — SELECT, WHERE, ORDER BY, LIMIT (Jour 1–2)

Anatomie d'une requête SQL

```
SELECT    colonnel1, colonne2          -- quoi afficher
FROM      nom_table                    -- d'où
WHERE     condition                     -- filtre (optionnel)
ORDER BY  colonne DESC                  -- tri (optionnel)
LIMIT     10;                          -- nb de lignes (optionnel)
```

Ordre d'exécution interne

L'ordre d'écriture est fixe. L'ordre d'exécution réel est différent : FROM → WHERE → SELECT → ORDER BY → LIMIT

Exercice 1.1 — Explorer coffee_sales

```
-- Aperçu général
SELECT * FROM coffee_sales LIMIT 10;

-- Quels types de paiement existent ?
SELECT DISTINCT cash_type FROM coffee_sales;

-- Quels cafés sont vendus ?
SELECT DISTINCT coffee_name FROM coffee_sales;
```

Équivalent Pandas :

```
df.head(10)
df['cash_type'].unique()
df['coffee_name'].unique()
```

Exercice 1.2 — Filtrer avec WHERE

```
-- Ventes en carte uniquement
SELECT * FROM coffee_sales
WHERE cash_type = 'card'
```

```

LIMIT 20;

-- Produits ayant rapporté plus de 50€ en une transaction
SELECT coffee_name, money FROM coffee_sales
WHERE money > 50;

-- Produits avec un prix entre 30 et 40€
SELECT coffee_name, money FROM coffee_sales
WHERE money BETWEEN 30 AND 40;

```

Équivalent Pandas :

```

df[df['cash_type'] == 'card']
df[df['money'] > 50]
df[df['money'].between(30, 40)]

```

Exercice 1.3 — Trier et isoler les top N

```

-- Les 10 transactions les plus élevées
SELECT coffee_name, money FROM coffee_sales
ORDER BY money DESC
LIMIT 10;

-- Films TMDB les mieux notés (avec minimum de votes)
SELECT title, vote_average, vote_count FROM tmdb_movies
WHERE vote_count > 1000
ORDER BY vote_average DESC
LIMIT 10;

```

Équivalent Pandas :

```

df.nlargest(10, 'money')[['coffee_name', 'money']]
tmdb[tmdb['vote_count'] > 1000].nlargest(10, 'vote_average')

```

Bloc 2 — GROUP BY et agrégats (Jour 2–3)

Notion clé : GROUP BY

```

SELECT  colonne_groupe,
        COUNT(*),          -- nombre de lignes
        SUM(colonne),      -- somme
        AVG(colonne),      -- moyenne
        MAX(colonne),      -- maximum

```

```

        MIN(colonne)          -- minimum
FROM      table
GROUP BY  colonne_groupe
ORDER BY  SUM(colonne) DESC;

```

Règle absolue

Toute colonne dans SELECT qui n'est pas une fonction d'agrégat DOIT être dans GROUP BY.

Exercice 2.1 — Top produits par chiffre d'affaires

```

SELECT
    coffee_name,
    ROUND(SUM(money), 2)      AS chiffre_affaires,
    COUNT(*)                  AS nb_ventes,
    ROUND(AVG(money), 2)     AS prix_moyen
FROM coffee_sales
GROUP BY coffee_name
ORDER BY chiffre_affaires DESC;

```

Équivalent Pandas :

```

df.groupby('coffee_name').agg(
    chiffre_affaires=('money', 'sum'),
    nb_ventes=('money', 'count'),
    prix_moyen=('money', 'mean')
).sort_values('chiffre_affaires', ascending=False)

```

Résultat attendu : Latte en tête sur le CA, Americano with Milk sur le volume.

Exercice 2.2 — Filtrer les produits > 100 ventes (HAVING)

```

SELECT
    coffee_name,
    COUNT(*)                  AS nb_ventes,
    ROUND(SUM(money), 2)     AS chiffre_affaires
FROM coffee_sales
GROUP BY coffee_name
HAVING COUNT(*) > 100
ORDER BY nb_ventes DESC;

```

Clause	Quand l'utiliser
WHERE	Filtre les lignes individuelles AVANT le regroupement
HAVING	Filtre les groupes APRÈS le GROUP BY

Exercice 2.3 — CA par mois

```
SELECT
    EXTRACT(MONTH FROM date)      AS mois,
    ROUND(SUM(money), 2)          AS chiffre_affaires,
    COUNT(*)                      AS nb_ventes
FROM coffee_sales
GROUP BY EXTRACT(MONTH FROM date)
ORDER BY mois;
```

Extractions temporelles disponibles :

Fonction SQL	Équivalent Pandas
EXTRACT(MONTH FROM date)	.dt.month
EXTRACT(YEAR FROM date)	.dt.year
EXTRACT(DOW FROM date)	.dt.dayofweek (0=dimanche en SQL)
TO_CHAR(date, 'Day')	.dt.day_name()
TO_CHAR(date, 'Month')	nom du mois en texte

Exercice 2.4 — Trafic Transilien par gare

```
SELECT
    gare,
    SUM(voyageurs)          AS total_voyageurs,
    COUNT(*)                AS nb_tranches,
    ROUND(AVG(voyageurs), 0) AS moyenne_par_tranche
FROM transilien
GROUP BY gare
ORDER BY total_voyageurs DESC
LIMIT 10;
```

Résultat attendu : Paris-Gare-du-Nord largement en tête — confirme l'analyse Pandas semaine 3.

Exercice 2.5 — Trafic Transilien par type de jour

```
SELECT
    type_jour,
    SUM(voyageurs)          AS total_voyageurs,
    ROUND(AVG(voyageurs), 0) AS moyenne,
    MAX(voyageurs)          AS pic_max
```

```
FROM transilien
GROUP BY type_jour
ORDER BY total_voyageurs DESC;
```

Bloc 3 — Jointures (Jour 4)

Les datasets actuels sont mono-table — pas de clé étrangère entre eux. L'objectif est de comprendre la logique JOIN avant le mois 2.

Notion JOIN

```
SELECT a.colonne1, b.colonne2
FROM   table_a AS a
JOIN   table_b AS b ON a.id = b.id;
```

Type	Ce qu'il retourne
INNER JOIN	Seulement les lignes qui matchent dans les deux tables
LEFT JOIN	Toutes les lignes de gauche + les matches de droite (NULL si absent)
RIGHT JOIN	Inverse du LEFT JOIN

Exercice pratique — Créer deux tables et les joindre

Création d'une table catalogue artificielle pour mettre en pratique le JOIN :

```
CREATE TABLE catalogue_cafe (
    coffee_name      VARCHAR(100),
    categorie         VARCHAR(50),
    prix_catalogue   NUMERIC(10, 2)
);

INSERT INTO catalogue_cafe VALUES
    ('Latte', 'Lait', 38.00),
    ('Americano with Milk', 'Americano', 32.00),
    ('Cappuccino', 'Lait', 35.00),
    ('Cortado', 'Espresso', 28.00),
    ('Cocoa', 'Spécial', 42.00),
    ('Espresso', 'Espresso', 22.00);

-- Joindre avec les ventes réelles
SELECT
```

```

    cs.coffee_name,
    cc.categorie,
    cc.prix_catalogue,
    ROUND (AVG (cs.money), 2)      AS prix_moyen_reel,
    COUNT (*)                      AS nb_ventes
FROM coffee_sales AS cs
JOIN catalogue_cafe AS cc ON cs.coffee_name = cc.coffee_name
GROUP BY cs.coffee_name, cc.categorie, cc.prix_catalogue
ORDER BY nb_ventes DESC;

```

Mini-projet final (Jours 4–5)

Objectif

Réaliser une mini-analyse complète sur Coffee Sales — en SQL pur, sans Pandas. Le résultat doit répondre à 3 questions métier.

Question 1 — Quels produits sont les plus rentables ?

```

SELECT
    coffee_name,
    COUNT (*)                      AS nb_ventes,
    ROUND (SUM (money), 2)        AS chiffre_affaires,
    ROUND (AVG (money), 2)        AS prix_moyen,
    ROUND (SUM (money) * 100.0
          / SUM (SUM (money)) OVER (), 1) AS part_ca_pct
FROM coffee_sales
GROUP BY coffee_name
ORDER BY chiffre_affaires DESC;

```

Notion nouvelle : `SUM(SUM(money)) OVER ()` est une window function — elle calcule le total général sans `GROUP BY` supplémentaire. À retenir comme outil existant, pas besoin de maîtriser maintenant.

Question 2 — Comment évoluent les ventes dans le temps ?

```

SELECT
    EXTRACT (YEAR FROM date)      AS annee,
    EXTRACT (MONTH FROM date)     AS mois,
    TO_CHAR (date, 'Month')       AS nom_mois,
    COUNT (*)                     AS nb_ventes,
    ROUND (SUM (money), 2)        AS chiffre_affaires

```



```

FROM coffee_sales
GROUP BY
    EXTRACT(YEAR FROM date),
    EXTRACT(MONTH FROM date),
    TO_CHAR(date, 'Month')
ORDER BY annee, mois;

```

Question 3 — Quel est le profil de la clientèle selon le jour ?

```

SELECT
    TO_CHAR(date, 'Day')           AS jour_semaine,
    EXTRACT(DOW FROM date)        AS num_jour,
    COUNT(*)                      AS nb_ventes,
    ROUND(SUM(money), 2)          AS chiffre_affaires,
    ROUND(AVG(money), 2)          AS panier_moyen
FROM coffee_sales
GROUP BY TO_CHAR(date, 'Day'), EXTRACT(DOW FROM date)
ORDER BY num_jour;

```

Requête de synthèse finale

```

-- Vue d'ensemble : produit × mois
SELECT
    coffee_name,
    EXTRACT(MONTH FROM date)      AS mois,
    COUNT(*)                     AS nb_ventes,
    ROUND(SUM(money), 2)         AS ca_mensuel
FROM coffee_sales
GROUP BY coffee_name, EXTRACT(MONTH FROM date)
ORDER BY coffee_name, mois;

```

Bilan Semaine 4 — Table de correspondance Pandas ↔ SQL

Pandas	SQL
pd.read_csv()	COPY table FROM 'fichier.csv' CSV HEADER;
df.head(10)	SELECT * FROM table LIMIT 10;
df.shape / len(df)	SELECT COUNT(*) FROM table;
df['col'].unique()	SELECT DISTINCT col FROM table;
df[df['col'] > x]	WHERE col > x
df.groupby('col').sum()	GROUP BY col + SUM(col)
sort_values('col', ascending=False)	ORDER BY col DESC
df.nlargest(5, 'col')	ORDER BY col DESC LIMIT 5
.dt.month	EXTRACT(MONTH FROM col)
.dt.year	EXTRACT(YEAR FROM col)
filtre post-groupby	HAVING

Points de vigilance

HAVING vs WHERE

Règle

Ne jamais utiliser WHERE pour filtrer un résultat de GROUP BY — utiliser HAVING.

Ordre des clauses — fixe et obligatoire

```
SELECT → FROM → WHERE → GROUP BY → HAVING → ORDER BY → LIMIT
```

Écrire dans un autre ordre provoque une erreur de syntaxe.

Les alias (AS) ne sont pas disponibles dans WHERE

```
-- INCORRECT
SELECT SUM(money) AS ca FROM coffee_sales WHERE ca > 1000;

-- CORRECT
SELECT SUM(money) AS ca FROM coffee_sales HAVING SUM(money) > 1000;
```

Format de documentation — Semaine 4

Conserver le format 3 questions introduit en semaine 1 :

Section	Contenu
Ce que j'ai fait	Calculé le chiffre d'affaires total par produit en SQL.
Pourquoi je l'ai fait	Retransposer l'analyse groupby() Pandas — vérifier que les résultats sont identiques.
Ce que ça m'a appris	GROUP BY en SQL suit la même logique que groupby() Pandas mais la syntaxe impose un ordre strict des clauses.

Semaine 4 terminée → **Mois 1 bouclé**